

Module 8

Capstone – End-to-End Verification & VIP Delivery (SV/UVM)

Learning objectives

- Plan and execute a ****single, substantial capstone project**** that exercises the full verification li...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

A["Topics"]

A --> N["Next module in course"]

Plan and execute a ****single, substantial capstone project**** that exercises the full verification lifecycle (Modules 1–7), from requirements...

Overview

- Module 8 is not another theory module; it is a
 major project that ties everything together:

How to learn this module

Suggested learning path

- Select capstone DUT and scope in
`module8/CAPSTONE\_PROJECT.md`.
- Scaffold Module 8 workspace: `./scripts/module8.sh
--scaffold`
- Assemble artifacts from Modules 1–7: plans, env,
coverage, regression ops, VIP (if applicable).
- Run full regression tiers and document sign-off
status in the capstone checklist.
- Validate deliverable completeness:
`./scripts/module8.sh --check`

Design architecture

1. Capstone system architecture

- `CAPSTONE\_PROJECT.md` defines DUT hierarchy, interfaces, clocks, and verification scope.
- Apply Modules 1–7 artifacts: plans, env, VIP (if applicable), coverage, regression ops.
- Deliverable layout: RTL, UVM env, tests, scripts, and user-facing documentation.

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
    subgraph DUT["stream_fifo (common_dut/rtl)"]
        SRC["Source interface\ns_valid / s_ready / s_data"]
        MEM["Circular buffer\nmem[DEPTH], wr_ptr, rd_ptr"]
        SNK["Sink interface\nm_valid / m_ready / m_data"]
        STS["Status\nlevel, overflow, underflow"]
    end
```

2. End-to-end verification package

- Single coherent verification solution or VIP another team can integrate.
- Traceability matrix from requirements through tests, coverage, and sign-off criteria.

```
Verification Architecture  
  
Diagram: Verification Architecture  
(render with mmdc when available)  
flowchart TB  
CAP["CAPSTONE_PROJECT.md"] --> REQ["Requirements"]  
REQ --> PLAN["Plans M1-M7"]  
PLAN --> RTL["DUT RTL"]  
PLAN --> TB["UVM env + VIP"]  
TB --> DEL["Deliverable package\ncode + docs + regressions"]
```

3. Capstone delivery and handoff workflow

- Plan: consolidate Modules 1–2 plans into capstone scope and test/regression strategy.
- Build: mature env, checkers, coverage, and VIP (Module 4–7 artifacts) under `rtl/` and `tb/`.
- Execute: run sanity → core → stress tiers; merge coverage; triage failures with saved seeds.
- Sign off: meet exit criteria in `CAPSTONE\_PROJECT.md`; complete `CHECKLIST.md`.
- Deliver: package RTL, UVM env, tests, scripts, and user docs for another team to integrate.

Capstone project plan template

```
# Capstone Project Plan (Module 8)
```

```
> **Purpose**: Define and track your major end-to-end verification or  
VIP project.
```

```
> **Module**: 8 – Capstone: End-to-End Verification & VIP Delivery  
(SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from  
`module8/templates/` if needed, or edit the file in `module8/`  
directly.
```

```
Reference: `module8/.solutions/`.
```

```
## 1. Capstone Choice
```

```
<!-- TODO: Project ID/Name, brief description, type  
(block/subsystem/VIP). -->
```

```
## 2. Requirements and Goals
```

Key files to study

Open these in the repo

- ``module8/CAPSTONE_PROJECT.md`` — end-to-end project definition and deliverables
- ``module8/CHECKLIST.md`` — capstone sign-off and delivery checklist
- ``docs/FILL_GUIDES.md`` — guidance for completing planning artifacts
- Prior module workspaces (``module1/`` ... ``module7/``) — reusable verification assets
- ``scripts/module8.sh`` — capstone documentation and checklist validation

Verification & testing methods

1. Full-lifecycle testing

- Execute planned regression tiers with coverage merge and documented closure status.
- Include negative, stress, and protocol/rule tests appropriate to your capstone DUT.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

M1["Planning"] --> M2["Tests"]

M2 --> M3["Coverage"]

M3 --> M4["Env / checkers"]

M4 --> M5["Regressions"]

M5 --> M6["Multi-agent"]

2. Sign-off and delivery testing

- Define explicit exit criteria (coverage %, bug bar, stability window).
- Retrospective: what worked, gaps, and improvements for a real project review.

3. Validation

- ``./scripts/module8.sh --check`` validates capstone documentation and checklist completion.
- Optional demo or walkthrough recorded as part of the deliverable.

Capstone checklist excerpt

Module 8 Checklist – Capstone: End-to-End Verification & VIP Delivery

> ****Purpose****: Track completion of your major capstone project.

> ****Instructions****: Check off items as you complete them. Copy from `module8/templates/` if needed, or edit the file in `module8/` directly.

Reference: `module8/.solutions/`.

1. Capstone Definition

- [] Capstone project chosen and described in `CAPSTONE\_PROJECT.md`.
- [] Key requirements and success criteria documented.
- [] Scope (block/subsystem/VIP) clearly defined and realistic for the time frame.

2. Re-use of Earlier Modules

Capstone validation script

```
#!/usr/bin/env bash

# Module 8: Capstone Orchestrator
# This script summarizes and checks the artifacts for the Module 8
# capstone.
# Usage: ./scripts/module8.sh [OPTIONS]

set -euo pipefail

# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
```

Command reference highlights

Scaffold and validate capstone

- `./scripts/module8.sh --scaffold`

Review capstone templates

- `head -45 module8/templates/CAPSTONE_PROJECT.md`

Hands-on examples

Module 8 self-check

```
$ ./scripts/module8.sh --help
```

Terminal: module8_check

```
[[0;36m===== [[0m
[[0;36mModule 8: Capstone - End-to-End Verification & VIP Delivery [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module8.sh [OPTIONS]

Module 8: Capstone - End-to-End Verification & VIP Delivery (SV/UVM)

This script summarizes and checks the capstone planning artifacts for Module 8.

OPTIONS:

--capstone-status	Show status of capstone_project.md only
--checklist-status	Show status of checklist_module8.md only
--summary	Show both statuses (default)
--check	Media/CI self-check (structure only)
--demo	Print example commands from EXAMPLES.md
--scaffold	Copy templates/*.md into module8/ if missing
--help, -h	Show this help message

EXAMPLES:

```
# Full summary
./scripts/module8.sh
```

```
# Focus only on capstone project plan
./scripts/module8.sh --capstone-status
```

Demo: Capstone project template

```
$ head -35 module8/templates/CAPSTONE_PROJECT.md
```

Terminal: ex01_capstone

Capstone Project Plan (Module 8)

> **Purpose**: Define and track your major end-to-end verification or VIP project.

> **Module**: 8 – Capstone: End-to-End Verification & VIP Delivery (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module8/templates/` if nee

1. Capstone Choice

<!-- TODO: Project ID/Name, brief description, type (block/subsystem/VIP). -->

2. Requirements and Goals

2.1 Functional Requirements (Draft)

<!-- TODO: Table of Req ID, Description, Priority, Notes. Link to REQUIREMENTS_MATRIX.md when us

2.2 Non-Functional / Quality Goals

<!-- TODO: Functional coverage, coverage targets, regression stability. -->

3. Re-use of Earlier Modules

<!-- TODO: List which earlier artifacts you are reusing or extending (Modules 1-7). -->

4. Architecture and Environment

<!-- TODO: DUT description, env and agents (UVM), integration. Textual architecture sketch. Code

5. Test Strategy and Regression

<!-- TODO: Tiers/jobs covering this capstone; new capstone-specific jobs; key test IDs and scena

6. Coverage Strategy

Demo: Capstone checklist

```
$ head -25 module8/templates/CHECKLIST.md
```

Terminal: ex02_checklist

Module 8 Checklist – Capstone: End-to-End Verification & VIP Delivery

> **Purpose**: Track completion of your major capstone project.

> **Instructions**: Check off items as you complete them. Copy from `module8/templates/` if need

1. Capstone Definition

- [] Capstone project chosen and described in `CAPSTONE\_PROJECT.md`.
- [] Key requirements and success criteria documented.
- [] Scope (block/subsystem/VIP) clearly defined and realistic for the time frame.

2. Re-use of Earlier Modules

- [] Existing plans (verification, test, coverage, regression) referenced and extended, not rei
- [] UVM env/checker designs from Modules 4-6 reused or evolved.
- [] Regression and orchestration concepts from Module 5 applied.

3. Implementation Progress (High-Level)

- [] Environment / VIP skeleton implemented in code.
- [] Core tests running and passing on the chosen DUT/IP.
- [] At least one coverage-enabled regression run executed.

4. Evidence of Full Lifecycle

Demo: Fill guides

```
$ head -30 docs/FILL_GUIDES.md
```

Terminal: ex03_fill_guides

Fill Guides - How to Complete Planning Documents

This document gives **section-by-section guidance** for filling the planning templates. Use it to

Table of Contents

- [Minimal vs full](#minimal-vs-full)
- [Module 1](#module-1)
 - [Verification Plan](#module-1---verification-plan)
 - [Requirements Matrix](#module-1---requirements-matrix)
 - [Checklist](#module-1---checklist)
 - [High-Priority Requirements Traceability](#module-1---high-priority-requirements-traceability)
- [Module 2](#module-2)
- [Modules 3-8](#modules-3-8)

Minimal vs full

Two ways to complete planning documents:

| Approach | What it means | Where to find it |
|----------------|--|------------------|
| Minimal | Enough to pass the scripts : no TODOs left, required sections present, at least | |
| Full | Full depth for review and sign-off: detailed DUT description, risk table, full | |

Recommendation: Start with a minimal pass so the script passes, then deepen section by section

Demo: Reference capstone

```
$ head -20 module8/.solutions/CAPSTONE_PROJECT.md
```

Terminal: ex04_solutions

Capstone Project Plan (Module 8)

> **Purpose**: Define and track your major end-to-end verification or VIP project.
> **Module**: 8 – Capstone: End-to-End Verification & VIP Delivery (SV/UVM)

1. Capstone Choice

- **Project ID / Name**: `CAP\_DMA\_FIFO`
- **Brief Description**:
- Verify a simple **DMA engine** that uses the shared `stream\_fifo` DUT as its internal buffer
- The DMA engine reads descriptors from a register interface, transfers a programmable number
- The capstone applies the full verification lifecycle (Modules 1-7) to this DMA, including re
- **Type**:
- [x] Block/IP verification
- [] Subsystem / SoC-style integration
- [] Protocol VIP (reusable agent + checker + coverage)

2. Requirements and Goals

Summarize the key requirements and goals (link to `REQUIREMENTS\_MATRIX.md` when used).

Demo: Module validation

```
$ ./scripts/module8.sh --check
```

Terminal: ex05_validate

```
[0;36m===== [0m  
[0;36mModule 8: Capstone - End-to-End Verification & VIP Delivery [0m  
[0;36m===== [0m
```

Module 8: media self-check
OK: module8/ exists
OK: templates/ exists
OK: EXAMPLES.md exists
OK: docs/MODULE8.md exists
OK: common_dut/rtl/ exists

All required checks passed.

Practice & assessment

What you should know (1/2)

- Drive an end-to-end verification effort from requirements → plan → env → tests → coverage → regress...
- Deliver a coherent, documented verification package or VIP that others can understand and use.
- Reflect on trade-offs and improvements as if you were preparing for a real project review.
- A clearly defined capstone project documented in ``capstone_project.md``.
- Evidence that Modules 1–7 have been applied (plans, env, tests, coverage, regressions).
- A “deliverable” verification solution or VIP (code + docs) that could be handed to another team.

What you should know (2/2)

- A short written or presented retrospective on what worked, what didn't, and what you'd improve next.

Assessment checklist

- A clearly defined capstone project documented in ``capstone_project.md``.
- Evidence that Modules 1–7 have been applied (plans, env, tests, coverage, regressions).
- A “deliverable” verification solution or VIP (code + docs) that could be handed to another team.
- A short written or presented retrospective on what worked, what didn’t, and what you’d improve next.

Summary & next steps

- Plan and execute a ****single, substantial capstone project**** that exercises the full verification li...
- Complete module8/CHECKLIST.md
- Review module8/EXAMPLES.md and run each lab
- Next: Next module in course